



सत्यं शिवं सुन्दरम्

Faculty of Technology & Engineering
The Maharaja Sayajirao University of Baroda

PROJECT REPORT

FS BE-III MICROPROCESSOR AND MICROCONTROLLER

Interface 8-digit 7 Seg LED display and 8 keys. Display 8-digit message in 8 different ways.

GUIDED BY:

(Prof.) Dr. Sunil Patel Sir

DONE BY:

RIDHAM NUVAL (8023055705)

Seat No. : 586519

SHLOK MISTRY (8023055581)

Seat No. : 586516

MAYUR PANDITA (8023057549)

Seat No. : 586527

ACADEMIC YEAR: 2025-26

AIM

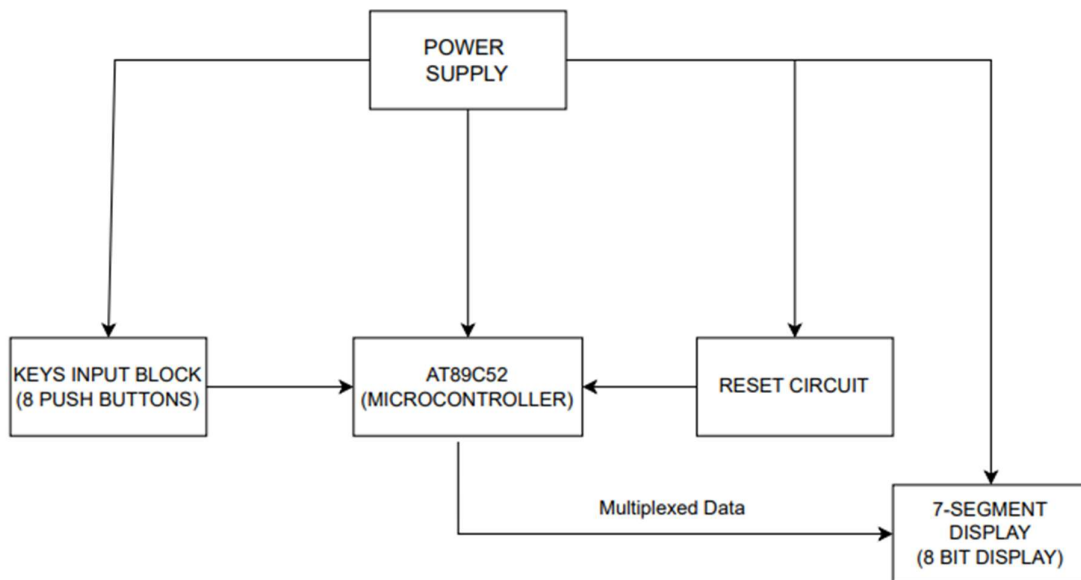
Interface 8-digit 7 Seg LED display and 8 keys. Display 8-digit message in 8 different ways as follows as per key pressed:

- (key-1) Steady message
- (key-2) Blinking message
- (key-3) Rolling through left
- (key-4) Rolling through right
- (key-5) Odd no LED's blinking others steady
- (key-6) Even no LED's blinking other's steady
- (key-7) Rolling half message from outside to inside
- (key-8) Rolling half message from inside to outside

LIST OF COMPONENTS

Sr. No.	Components	Quantity
1	AT89C52 40 Pin IC Base	1
2	Push Button (5mm)	8
3	7-Seg LED Display(Common Anode)	8
4	USB Connector	1
5	1000 μ F/35V Capacitor	1
6	33 pF/25V Capacitor	2
7	470 μ F/25V Capacitor	1
8	10 μ F/35V Capacitor	1
9	104 Capacitor	3
10	330E Resistors	9
11	10k Resistors	8
12	4k7 Resistors	1
13	IC 7805 (Voltage Regulator 5V)	1
14	BC558 (PNP) Transistors	8
15	IN4148 Diode (Zener)	1
16	IN4007 Diode	5
17	LED (5mm)	1
18	12MHz Crystal	1
19	10k, 9 Pin R-Pack	1

BLOCK DIAGRAM



BLOCK DESCRIPTIONS

1. Power Supply

- Provides the required +5V regulated DC supply to the microcontroller (AT89C52), display, and input circuit.
- Typically consists of:
 - Step-down transformer (230V AC → 12V AC),
 - Bridge rectifier (AC → DC),
 - Filter capacitor (to remove ripples),
 - Voltage regulator (7805) to maintain +5V stable output.
- Ensures stable operation of logic circuits and LEDs.

2. Reset Circuit

- Provides a power-on reset to the AT89C52 microcontroller.
- Consists of a resistor-capacitor (RC) network and a push-button.
- On power-up, it ensures the microcontroller starts execution from the reset vector (0000H).
- Manual reset option allows the user to restart the program anytime.

3. Keys Input Block (8 Push Buttons)

- 8 push buttons are connected to one port of the microcontroller.
- Each key corresponds to one display function:
 - Key-1: Steady message
 - Key-2: Blinking message
 - Key-3: Rolling left
 - Key-4: Rolling right
 - Key-5: Odd digits blink, even steady
 - Key-6: Even digits blink, odd steady
 - Key-7: Half message rolls from outside → inside
 - Key-8: Half message rolls from inside → outside
- The microcontroller continuously scans the keys (polling or interrupt method) and selects the display mode accordingly.

4. AT89C52 Microcontroller

- Acts as the control unit of the system.
- Main tasks:
 - Reads the pressed key from the input block.
 - Fetches the stored 8-digit message (in code memory).
 - Generates multiplexed signals for the 7-segment display.
 - Applies the selected effect (steady, blinking, rolling, etc.).
- Features of AT89C52 relevant here:
 - 4 I/O ports → can handle display multiplexing and input.
 - Internal timers used for delay generation (blinking, rolling).
 - Fast control logic to refresh display and maintain persistence of vision.

5. 7-Segment Display (8-Digit, Multiplexed)

- Consists of 8 individual 7-segment LED modules arranged side by side.
- Displays 8-digit alphanumeric message.
- Multiplexing is used:
 - Only one digit glows at a time, but switching is fast.
 - To human eyes, it appears as if all digits glow simultaneously.
- Controlled by:
 - Segment lines (a–g, DP) connected commonly.
 - Digit enable lines (through transistors or drivers like ULN2803).
- The microcontroller updates display continuously to show effects like blinking or rolling.

Working (Overall Flow)

1. User presses a key → microcontroller detects it.
2. Microcontroller selects the corresponding display mode.
3. Message is stored in program memory → sent to display driver logic.
4. Using multiplexing technique, display is refreshed at high speed.
5. Depending on the mode:
 - Steady → fixed display.
 - Blinking → alternates between ON and OFF.
 - Rolling → shifts message left or right.
 - Odd/Even blinking → selective digits blink.
 - Outside → inside roll → message parts shift toward center.
 - Inside → outside roll → message parts shift outward.

FILE NAME: 8051 project.pdsprj
 DESIGN TITLE: 8051 project.pdsprj
 PATH: C:\Users\Shlok\Desktop\8051 Project\8051 project.pdsprj
 BY: @AUTHOR
 BOARD: @BOARD

DATE: 21-09-2025
 1 of 1
 TIME: 09:28:25

8051 consists of 4 PORTS out of which:

- PORT 0 is connected to the Pins of 7 Segment LEDs in Common Anode Configuration via a 330Ω resistors on the Data Bus.
- PORT 2 acts as Segment Drive consisting of 8 PNP transistors whose Collectors are connected to Common Anode of the 7 Segment LEDs, Emitters are connected to a common Ground and Base are connected to 8 different Pins in Port 2 via 10k registers.
- PORT 3 is free for future use.
- PORT 1 contains 8 individual keys, each used for displaying different message:
 - Key 1-Steady message
 - Key 2-Blinking message
 - Key 3-Rolling through right
 - Key 4-Rolling through left
 - Key 5-Even no LEDs blinking other's steady
 - Key 6-Odd LEDs blinking other's steady
 - Key 7-Rolling half message from outside to inside

- Key 8-Rolling half message from inside to outside. Keys are connected in Pull Down configuration given 5 V common supply connected to 10k register pack.
- Crystal Oscillator is connected to Pins XTAL 1 and XTAL 2 with two 33pF capacitors in parallel representing the crystal circuit.
- Reset circuit is connected to Pin RST in Power ON Reset configuration.
- Input Supply is given through a 12V adapter which passes through a bridge rectifier and Regulator IC 7805 to get a 5V constant supply for the 8051 Microcontroller.

CODE

```

ORG 0000H                                ; Program starts at memory address 0000H
ACALL TEST0                              ; Call TEST0 subroutine

;--- MAIN PROGRAM LOOP ---
MAIN:
JB P1.0, NEXT_1                          ; Jump to NEXT_1 if P1.0 is high (not pressed)
ACALL MSG_1                              ; If P1.0 is low (pressed), call MSG_1
SJMP MAIN                                ; Loop back to MAIN

NEXT_1:
JB P1.1, NEXT_2                          ; Jump to NEXT_2 if P1.1 is high (not pressed)
ACALL MSG_2                              ; If P1.1 is low (pressed), call MSG_2

NEXT_2:
JB P1.2, NEXT_3                          ; Jump to NEXT_3 if P1.2 is high (not pressed)
ACALL MSG_3                              ; If P1.2 is low (pressed), call MSG_3

```

```

NEXT_3:
JB P1.3, NEXT_4          ; Jump to NEXT_4 if P1.3 is high (not pressed)
ACALL MSG_4              ; If P1.3 is low (pressed), call MSG_4

NEXT_4:
JB P1.4, NEXT_5          ; Jump to NEXT_5 if P1.4 is high (not pressed)
ACALL MSG_5              ; If P1.4 is low (pressed), call MSG_5

NEXT_5:
JB P1.5, NEXT_6          ; Jump to NEXT_6 if P1.5 is high (not pressed)
ACALL MSG_6              ; If P1.5 is low (pressed), call MSG_6

NEXT_6:
JB P1.6, NEXT_7          ; Jump to NEXT_7 if P1.6 is high (not pressed)
ACALL MSG_7              ; If P1.6 is low (pressed), call MSG_7

NEXT_7:
JB P1.7, MAIN            ; Jump to MAIN if P1.7 is high (not pressed)
ACALL MSG_8              ; If P1.7 is low (pressed), call MSG_8

SJMP MAIN                ; After MSG_8, loop back to the beginning of MAIN

;--- SUBROUTINES ---

TEST_1:
MOV R2, #08H            ; R2 counter for 8 digits/steps
MOV P0, #00H            ; P0 is the cathode driver, set to 00H (all on)
MOV R1, #00H            ; R1 counter for accessing array elements

```

```

REFRESH_1:
MOV DPTR ,#DIGIT0      ; Load DPTR with the address of DIGIT0 array
MOV A, R1               ; Load A with the current index from R1
MOVC A, @A+DPTR         ; Move code byte from (A + DPTR) to A (get digit pattern)
MOV P2, A               ; Display the pattern on Port 2
INC R1                  ; Increment R1 to get the next digit pattern
ACALL DELAY500MS        ; Wait for 500 milliseconds
DJNZ R2, REFRESH_1      ; Decrement R2 and loop back if not zero
MOV P0, #0FFH           ; Turn off all digits on Port 0
MOV P2, #0FFH           ; Turn off all segments on Port 2
RET                     ; Return from subroutine

```

```

MSG_1:
MOV R3, #70             ; R3 counter

MAIN_1:
MOV R2, #08             ; R2 counter for inner loop
MOV P0, #00H            ; P0 is the cathode driver, all on
MOV P2, #00H            ; P2 is the segment driver, all segments on

```

```

REFRESH_2:
ACALL DELAY1MS          ; Wait for 1 millisecond
JNB P1.0, NEXT_1        ; Check P1.0 again, jump to NEXT_1 if low
DJNZ R2, REFRESH_2      ; Decrement R2 and loop back if not zero
ACALL DELAY100MS        ; Delay for 100ms
RET                     ; Return from subroutine

```

```

MSG_2:
MAIN_2:
JNB P1.1, NEXT_2        ; Check P1.1, jump to NEXT_2 if high

```

MOV R2, #100	; R2 counter
MOV P0, #00H	; P0 all on
MOV P2, #00H	; P2 all on
ACALL DELAY500MS	; Wait 500ms
MOV P0, #0FFH	; P0 all off
ACALL DELAY500MS	; Wait 500ms
SJMP MAIN_2	; Loop back to MAIN_2 (infinite loop if P1.1 is low)
MOV P0, #0FFH	
MOV P2, #0FFH	
RET	
MSG_3:	
ACALL TEST_1	; Call TEST_1 subroutine
RET	; Return
MSG_4:	
MOV R2, #08H	; R2 counter for 8 digits/steps
MOV P0, #00H	; P0 all on
MOV R1, #00H	; R1 counter for array index
REFRESH_3:	
MOV DPTR, #DIGIT4	; Load DPTR with the address of DIGIT4 array
MOV A, R1	; Load A with the current index
MOVC A, @A+DPTR	; Get byte from code memory
MOV P2, A	; Display pattern on P2
INC R1	; Increment R1
ACALL DELAY500MS	; Wait 500ms
DJNZ R2, REFRESH_3	; Decrement R2 and loop back if not zero
MOV P0, #0FFH	; P0 all off
MOV P2, #0FFH	; P2 all off
RET	; Return

MSG_5:

MAIN_3:

```
MOV P0, #00H          ; P0 all on
MOV P2, #00H          ; P2 all on
ACALL DELAY500MS       ; Wait 500ms
MOV P2, #0AAH         ; Display pattern 0AAH (alternating bits) on P2
ACALL DELAY500MS       ; Wait 500ms
SJMP MAIN_3           ; Loop back to MAIN_3 (infinite loop)
MOV P0, #0FFH
MOV P2, #0FFH
RET
```

MSG_6:

MAIN_4:

```
MOV R2, #100          ; R2 counter
MOV P0, #00H          ; P0 all on
MOV P2, #00H          ; P2 all on
ACALL DELAY500MS       ; Wait 500ms
MOV P2, #55H          ; Display pattern 55H on P2
ACALL DELAY500MS       ; Wait 500ms
SJMP MAIN_4           ; Loop back to MAIN_4 (infinite loop)
MOV P0, #0FFH
MOV P2, #0FFH
RET
```

MSG_7:

MOV R2, #04H ; R2 counter for 4 digits/steps

MOV P0, #00H ; P0 all on

MOV R1, #00H ; R1 counter for array index

REFRESH_4:

MOV DPTR, #DIGIT7 ; Load DPTR with the address of DIGIT7 array

MOV A, R1 ; Load A with the current index

MOVC A, @A+DPTR ; Get byte from code memory

MOV P2, A ; Display pattern on P2

INC R1 ; Increment R1

ACALL DELAY1SEC ; Wait 1 second

DJNZ R2, REFRESH_4 ; Decrement R2 and loop back if not zero

MOV P0, #0FFH ; P0 all off

MOV P2, #0FFH ; P2 all off

RET ; Return

MSG_8:

MOV R2, #04H ; R2 counter for 4 digits/steps

MOV P0, #00H ; P0 all on

MOV R1, #00H ; R1 counter for array index

REFRESH_5:

MOV DPTR, #DIGIT8 ; Load DPTR with the address of DIGIT8 array

MOV A, R1 ; Load A with the current index

MOVC A, @A+DPTR ; Get byte from code memory

MOV P2, A ; Display pattern on P2

INC R1 ; Increment R1

ACALL DELAY1SEC ; Wait 1 second

DJNZ R2, REFRESH_5 ; Decrement R2 and loop back if not zero

```

MOV P0, #0FFH          ; P0 all off

MOV P2, #0FFH          ; P2 all off

RET                    ; Return

;--- DATA ARRAYS ---

DIGIT0: DB 0FEH, 0FCH, 0F8H, 0F0H, 0E0H, 0C0H, 080H, 00H ; Data for first 8-step display

DIGIT4: DB 07FH, 03FH, 01FH, 00FH, 007H, 003H, 001H, 00H

                                ;Data for another 8-step display

DIGIT7: DB 07EH, 03CH, 018H, 00H  ; Data for a 4-step display

DIGIT8: DB 0E7H, 0C3H, 081H, 00H ; Data for a 4-step display


;--- DELAY SUBROUTINES ---

DELAY1MS:

MOV R2, #4              ; Outer loop counter

L1: MOV R1, #250         ; Inner loop counter

L2: DJNZ R1, L2          ; Decrement R1 and loop until zero

DJNZ R2, L1             ; Decrement R2 and loop until zero

RET                    ; Return


DELAY100MS:

MOV TMOD, #01H          ; Configure Timer 0 for mode 1 (16-bit timer)

MOV TH0, #0B1H          ; Load high byte of timer register

MOV TL0, #0E0H          ; Load low byte of timer register

SETB TR0               ; Start Timer 0

WAIT1: JNB TF0, WAIT1   ; Wait for timer overflow flag TF0 to be set

CLR TR0                ; Stop Timer 0

CLR TF0                ; Clear the overflow flag

RET                    ; Return

```

DELAY300MS:

MOV R3, #3 ; Loop 3 times

DLY_LOOP:

ACALL DELAY100MS ; Call 100ms delay

DJNZ R3, DLY_LOOP ; Decrement R3 and loop

RET ; Return

DELAY500MS:

MOV R3, #5 ; Loop 5 times

DLY500_LOOP:

ACALL DELAY100MS ; Call 100ms delay

DJNZ R3, DLY500_LOOP ; Decrement R3 and loop

RET ; Return

DELAY1SEC:

MOV R3, #10 ; Loop 10 times

DLY1SEC_LOOP:

ACALL DELAY100MS ; Call 100ms delay

DJNZ R3, DLY1SEC_LOOP ; Decrement R3 and loop

RET ; Return

END ; End of program

NEW CODE

ORG 0000H ;initialising the program

MAIN: ; routine for setup

MOV R3,#00H ; digit displayer

MOV R4,#07H ; segment selector

MOV R5,#20H ; display loop for blinking

MOV R6,#10H ; segment selector for rolling

MOV P1,#0FFH ; setting port 1 as input

LJMP CHECK_BUTTON ; long jump to buttons

STEADY: ;steady message printing - 125 Hz refresh rate

MOV DPTR,#SEG_TABN

MOV A,R3

MOVC A,@A+DPTR

MOV P0,A

MOV DPTR,#SEG_TABS

MOV A,R4

MOVC A,@A+DPTR

MOV P2,A

LCALL SMALL_DELAY

MOV P2,#0FFH

INC R3

DEC R4

CJNE R3,#08H,STEADY

LCALL CHECKER

MOV R3,#00H

MOV R4,#07H

SJMP STEADY

BLINKING: ;blinking message with 500ms duty cycle

MOV DPTR,#SEG_TABN

MOV A,R3

MOVC A,@A+DPTR

MOV P0,A

MOV DPTR,#SEG_TABS

MOV A,R4

MOVC A,@A+DPTR

MOV P2,A

LCALL SMALL_DELAY

MOV P2,#0FFH

INC R3

DEC R4

CJNE R3,#08H,BLINKING

LCALL CHECKER

MOV R3,#00H

MOV R4,#07H

DJNZ R5,BLINKING

MOV R5,#20H ; time to keep the display on

MOV P0,#0FFH

MOV P2,#0FFH

LCALL MEDIUM_DELAY ; time to keep display off

SJMP BLINKING

CHECK_BUTTON: ; initiator of the display pattern using buttons

JNB P1.0,STEADY

JNB P1.1,BLINKING

JNB P1.2,ROLL_LEFT_MAIN

JNB P1.3,ROLL_RIGHT_MAIN

LJMP CHECK_BUTTON1 ; carry forward of display pattern initiator for other 4 button

ROLL_LEFT_MAIN: ; setup for left rolling message

MOV R3,#10H

MOV R6,#10H

ROLL_LEFT: ; message appearing from left side and vanishing
to right completely

MOV DPTR,#SEG_TABROLL

MOV A,R3

MOVC A,@A+DPTR

MOV P0,A

MOV DPTR,#SEG_TABS

MOV A,R4

MOVC A,@A+DPTR

MOV P2,A

LCALL SMALL_DELAY

MOV P2,#0FFH

DJNZ R5,ROLL_LEFT

MOV R5,#20H

INC R3 ;MSG

DEC R4;SEG

CJNE R4,#0FFH,ROLL_LEFT

LCALL CHECKER

MOV R4,#07H

DEC R6

MOV R3,06H

```
CJNE R6,#0FFH,ROLL_LEFT
```

```
SJMP ROLL_LEFT_MAIN
```

```
ROLL_RIGHT_MAIN:                ;setup for right rolling message
```

```
MOV R3,#00H
```

```
MOV R6,#00H
```

```
ROLL_RIGHT:                      ;message appearing from right side and  
                                vanishing to left completely
```

```
MOV DPTR,#SEG_TABROLL
```

```
MOV A,R3
```

```
MOVC A,@A+DPTR
```

```
MOV P0,A
```

```
MOV DPTR,#SEG_TABS
```

```
MOV A,R4
```

```
MOVC A,@A+DPTR
```

```
MOV P2,A
```

```
LCALL SMALL_DELAY
```

```
MOV P2,#0FFH
```

```
DJNZ R5,ROLL_RIGHT
```

```
MOV R5,#20H
```

```
INC R3 ;MSG
```

```
DEC R4;SEG
```

```
CJNE R4,#0FFH,ROLL_RIGHT
```

```
LCALL CHECKER
```

```
MOV R4,#07H
```

```
INC R6
```

```
MOV R3,06H
```

```
CJNE R6,#10H,ROLL_RIGHT
```

SJMP ROLL_RIGHT_MAIN

ODD_BLINK_MAIN: ;setup for odd digits blink

MOV R5,#20H

MOV R3,#00H

MOV R4,#07H

;message with odd digits blinking rest steady

ODD_BLINK: ;steady message printing for 500ms

MOV DPTR,#SEG_TABN

MOV A,R3

MOVC A,@A+DPTR

MOV P0,A

MOV DPTR,#SEG_TABS

MOV A,R4

MOVC A,@A+DPTR

MOV P2,A

LCALL SMALL_DELAY

MOV P2,#0FFH

INC R3

DEC R4

CJNE R3,#08H,ODD_BLINK

LCALL CHECKER

MOV R3,#00H

MOV R4,#07H

DJNZ R5,ODD_BLINK

MOV R5,#20H

MOV R3,#01H

MOV R4,#06H

SJMP ODD_BLANK

ODD_BLANK: ;even digits on only for 500ms

MOV DPTR,#SEG_TABN

MOV A,R3

MOVC A,@A+DPTR

MOV P0,A

MOV DPTR,#SEG_TABS

MOV A,R4

MOVC A,@A+DPTR

MOV P2,A

LCALL SMALL_DELAY

MOV P2,#0FFH

INC R3

DEC R4

INC R3

DEC R4

CJNE R3,#09H,ODD_BLANK

LCALL CHECKER

MOV R3,#01H

MOV R4,#06H

DJNZ R5,ODD_BLANK

MOV R5,#20H

SJMP ODD_BLINK_MAIN

CHECK_BUTTON1: ;carry forwarded routine of checking button

JNB P1.4,ODD_BLINK_MAIN

JNB P1.5,EVEN_BLINK_MAIN

LJMP CHECK_BUTTON2 ;carry forward checking of button

EVEN_BLINK_MAIN:

MOV R5,#20H

;message with odd digits blinking rest steady

EVEN_BLINK: ;steady message printing for 500ms

MOV DPTR,#SEG_TABN

MOV A,R3

MOVC A,@A+DPTR

MOV P0,A

MOV DPTR,#SEG_TABS

MOV A,R4

MOVC A,@A+DPTR

MOV P2,A

LCALL SMALL_DELAY

MOV P2,#0FFH

INC R3

DEC R4

CJNE R3,#08H,EVEN_BLINK

LCALL CHECKER

MOV R3,#00H

MOV R4,#07H

DJNZ R5,EVEN_BLINK

MOV R5,#20H

SJMP EVEN_BLANK

EVEN_BLANK: ;odd digits on only for 500ms

MOV DPTR,#SEG_TABN

MOV A,R3

MOVC A,@A+DPTR

```

MOV P0,A
MOV DPTR,#SEG_TABS
MOV A,R4
MOVC A,@A+DPTR
MOV P2,A
LCALL SMALL_DELAY
MOV P2,#0FFH
INC R3
DEC R4
INC R3
DEC R4
CJNE R3,#08H,EVEN_BLANK
LCALL CHECKER
MOV R3,#00H
MOV R4,#07H
DJNZ R5,EVEN_BLANK
MOV R5,#20H
SJMP EVEN_BLINK

```

CHECK_BUTTON2: ;carry forwarded routine of checking button

```
JNB P1.6,OUT_IN_MAIN
```

```
JNB P1.7,IN_OUT_MAIN
```

```
LJMP CHECK_BUTTON;looping back to checking button
```

;message appearing from outside to inside

;first loop rolls message from left to right for 4 digits

;second loop rolls message from right to left for 4 digits

OUT_IN_MAIN: ;setup for outside to inside rolling message

;FIRST LOOP

MOV R3,#07H ;MSG

MOV R4,#07H ;SEG

MOV R6,#07H

;SECOND LOOP

MOV 30H,#0DH

MOV 31H,#03H

MOV 32H,#04H ;NO OF TIME THE LOOP SHOULD RUN

MSG_LOOP2 EQU 30H ;using internal ram program space for

SEG_LOOP2 EQU 31H ;defining variables

LOOP_RUNNER EQU 32H

MOV R7,#0DH

MOV R5,#20H

OUT_IN: ;start of animation

MOV DPTR,#SEG_TABNO

MOV A,R3

MOVC A,@A+DPTR

MOV P0,A

MOV DPTR,#SEG_TABS

MOV A,R4

MOVC A,@A+DPTR

MOV P2,A

LCALL SMALL_DELAY

MOV P2,#0FFH

MOV DPTR,#SEG_TABNO

MOV A,MSG_LOOP2

```

MOVC A,@A+DPTR
MOV P0,A
MOV DPTR,#SEG_TABS
MOV A,SEG_LOOP2
MOVC A,@A+DPTR
MOV P2,A
LCALL SMALL_DELAY
MOV P2,#0FFH
DJNZ R5,OUT_IN
MOV R5,#20H
INC R3 ;MSG
DEC R4;SEG
INC MSG_LOOP2      ;MSG
DEC SEG_LOOP2      ;SEG
MOV A,SEG_LOOP2
CJNE A,#0FFH,OUT_IN
LCALL CHECKER
MOV R4,#07H
DEC R6
MOV R3,06H
MOV SEG_LOOP2,#03H
INC R7
MOV MSG_LOOP2,07H
DJNZ LOOP_RUNNER,OUT_IN
LJMP STEADY

```

;message appearing from inside to outside

;first loop rolls message from right to left for 4 digits

;second loop rolls message from left to right for 4 digits

IN_OUT_MAIN: ;setup for outside to inside rolling message

;FIRST LOOP

MOV R3,#14H

MOV R4,#03H

MOV R6,#14H

;SECOND LOOP

MOV 33H,#01H

MOV 34H,#07H

MOV 35H,#04H ;NO OF TIME THE LOOP SHOULD RUN

MSG_LOOP21 EQU 33H

SEG_LOOP21 EQU 34H

LOOP_RUNNER1 EQU 35H

MOV R7,#01H

MOV R5,#20H

IN_OUT:

MOV DPTR,#SEG_TABNO

MOV A,R3

MOVC A,@A+DPTR

MOV P0,A

MOV DPTR,#SEG_TABS

MOV A,R4

MOVC A,@A+DPTR

MOV P2,A

LCALL SMALL_DELAY

MOV P2,#0FFH

MOV DPTR,#SEG_TABNO

```
MOV A,MSG_LOOP21
MOVC A,@A+DPTR
MOV P0,A
MOV DPTR,#SEG_TABS
MOV A,SEG_LOOP21
MOVC A,@A+DPTR
MOV P2,A
LCALL SMALL_DELAY
MOV P2,#0FFH
DJNZ R5,IN_OUT
MOV R5,#20H
INC R3 ;MSG
DEC R4;SEG

INC MSG_LOOP21    ;MSG
DEC SEG_LOOP21    ;SEG
MOV A,SEG_LOOP21
CJNE A,#03H,IN_OUT

LCALL CHECKER
MOV R4,#03H
DEC R6
MOV R3,06H
MOV SEG_LOOP21,#07H
INC R7
MOV MSG_LOOP21,07H
DJNZ LOOP_RUNNER1,IN_OUT
LJMP STEADY
```

CHECKER: ;routine which constantly checks if any button is pressed
or not

JNB P1.0,CHECK_BUTTONS

LCALL SMALL_DELAY

LCALL SMALL_DELAY

JNB P1.1,CHECK_BUTTONS

LCALL SMALL_DELAY

LCALL SMALL_DELAY

JNB P1.2,CHECK_BUTTONS

LCALL SMALL_DELAY

LCALL SMALL_DELAY

JNB P1.3,CHECK_BUTTONS

LCALL SMALL_DELAY

LCALL SMALL_DELAY

JNB P1.4,CHECK_BUTTONS

LCALL SMALL_DELAY

LCALL SMALL_DELAY

JNB P1.5,CHECK_BUTTONS

LCALL SMALL_DELAY

LCALL SMALL_DELAY

JNB P1.6,CHECK_BUTTONS

LCALL SMALL_DELAY

LCALL SMALL_DELAY

JNB P1.7,CHECK_BUTTONS

LCALL SMALL_DELAY

LCALL SMALL_DELAY

RET

```
CHECK_BUTTONS:           ;confirms the press status of button
    MOV P0,#0FFH          ;no data to display
    MOV P2,#0FFH          ;turns off the digits
    MOV R3,#00H           ;resetting values for next display
    MOV R4,#07H
    LJM MAIN    ;long jump to starting condition of display type
```

; --- Delay Routines ---

```
SMALL_DELAY:              ;few microseconds delay
    MOV R0,#0FFH
    DEL:DJNZ R0,DEL
    RET
```

```
DELAY:                    ;one second delay
    MOV R0,#0FFH
    MOV R1,#0FFH
    MOV R2,#09H
    LOOP1: DJNZ R0,LOOP1
    LOOP2: MOV R0,#0FFH
    DJNZ R1,LOOP1
    LOOP3: MOV R1,#0FFH
    DJNZ R2,LOOP1
    RET
```

```
MEDIUM_DELAY:            ;0.5 second delay
    MOV R0,#0FFH
    MOV R1,#0FFH
```

```

MOV R2,#04H

LOOP11: DJNZ R0,LOOP11

LOOP12: MOV R0,#0FFH

DJNZ R1,LOOP11

LOOP13: MOV R1,#0FFH

DJNZ R2,LOOP11

RET

; --- Lookup Table (message is 'BE-3uPuC', common anode)
089H,086H,091H,0BFH,080H,0C0H,092H,0F9H---

SEG_TABN: ;message table 'BE3-uPuC'

DB 00H,06H,30H,0BFH,63H,0CH,63H,0C6H

SEG_TABROLL: ;message for rolling

DB
0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,00H,06H,30H,0BFH,63H,0CH,63H,0C6H,
0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH

SEG_TABNO: ;message for rolling OUT-IN-OUT

DB
0FFH,0FFH,0FFH,0FFH,00H,06H,30H,0BFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0F
FH,63H,0CH,63H,0C6H,0FFH,0FFH,0FFH,0FFH

SEG_TABS: ;individual segment turn on from right to left

DB 0FEH,0FDH,0FBH,0F7H,0EFH,0DFH,0BFH,07FH

END—

```

PCB LAYOUT :

